

Aufgabe 1 (Symbolische vs. Non-Symbolische KI).

Betrachten Sie die folgenden beiden Szenarien:

Szenario 1: Ein Expertensystem wird entwickelt, das mithilfe einer Reihe von festgelegten Regeln diagnostische Entscheidungen in der Medizin trifft. Es verwendet explizite Logikregeln, um Symptome zu analysieren und entsprechende Diagnosen zu stellen

Szenario 2: Ein neuronales Netzwerk wird trainiert, um anhand von Tausenden von Bildern von Hautläsionen die Wahrscheinlichkeit für das Vorliegen von Hautkrebs zu bestimmen. Das Netzwerk lernt durch das Anpassen seiner Gewichte während des Trainingsprozesses, Muster in den Bildern zu erkennen.

1a) Ordnen Sie die beiden Szenarien entweder der **symbolischen** oder der **non-symbolischen KI** zu und begründen Sie Ihre Zuordnung.

1b) Diskutieren Sie die Vor- und Nachteile der symbolischen und nonsymbolischen KI, insbesondere in Bezug auf ihre Anwendbarkeit und Interpretierbarkeit.

Aufgabe 2 (Erste Schritte mit Python und Numpy)

NumPy (Numerical Python) ist die grundlegende Bibliothek für wissenschaftliches Rechnen in Python. Sie bietet:

- Leistungsstarke n-dimensionale Arrays
- Mathematische Funktionen für lineare Algebra, Statistik, etc.
- Hohe Performance durch C-optimierte Operationen
- Grundlage für viele Data-Science-Bibliotheken (pandas, matplotlib, SciPy, scikit-learn, ...)

Falls Sie nicht mit Python vertraut sind, hier finden Sie ein einfaches Tutorial:

<https://docs.python.org/3/tutorial/> Grundlegendes Verständnis von Python wird im folgenden vorausgesetzt.

```
In [ ]: import numpy as np # Standard-Import, der Name np ist Konvention
```

Arrays erstellen

```
In [ ]: # Aus Python-Listen
arr1 = np.array([1, 2, 3, 4, 5])
print("1D-Array:", arr1)

# Mehrdimensional
arr2 = np.array([[1, 2, 3], [4, 5, 6]])
print("2D-Array:\n", arr2)

# Besondere Arrays
zeros = np.zeros((3, 3))      # Mit Nullen
ones = np.ones((2, 4))       # Mit Einsen
range_arr = np.arange(0, 10, 2) # Wie range(), aber als Array
random_arr = np.random.rand(5) # Zufallszahlen

print("Nullen:\n", zeros)
print("Zufall:", random_arr)
```

Array-Eigenschaften

```
In [ ]: arr = np.array([[1, 2, 3], [4, 5, 6]])

print("Shape:", arr.shape)      # Dimensionen: (2, 3)
print("Anzahl Dimensionen:", arr.ndim) # 2
print("Anzahl Elemente:", arr.size) # 6
print("Datentyp:", arr.dtype)   # int64
```

Array-Operationen

```
In [ ]: a = np.array([1, 2, 3])
b = np.array([4, 5, 6])

# Elementweise Operationen
print("Addition:", a + b)      # [5 7 9]
print("Multiplikation:", a * b) # [4 10 18]
print("Potenz:", a ** 2)       # [1 4 9]

# Vergleichsoperationen
print("Vergleich:", a > 2)     # [False False True]
```

Mathematische Funktionen

```
In [ ]: arr = np.array([1, 2, 3, 4, 5])

print("Summe:", np.sum(arr))    # 15
print("Mittelwert:", np.mean(arr)) # 3.0
print("Maximalwert:", np.max(arr)) # 5
print("Standardabweichung:", np.std(arr)) # 1.414

# Trigonometrie
angles = np.array([0, np.pi/2, np.pi])
print("Sinus:", np.sin(angles)) # [0. 1. 0.]
```

Array-Indizierung und Slicing

```
In [ ]: arr = np.array([1, 2, 3, 4, 5, 6, 7, 8, 9, 10])

# Filtern mit Bedingungen
even_numbers = arr[arr % 2 == 0]
big_numbers = arr[arr > 5]

print("Gerade Zahlen:", even_numbers) # [2,4,6,8,10]
print("Zahlen > 5:", big_numbers)    # [6,7,8,9,10]
```

Shape Manipulation

```
In [ ]: arr = np.arange(12) # [0,1,2,3,4,5,6,7,8,9,10,11]

# Umformen
matrix = arr.reshape(3, 4)
print("Umgeformt (3x4):\n", matrix)

# Transponieren
transposed = matrix.T
print("Transponiert (4x3):\n", transposed)

# Flach machen
flattened = matrix.flatten()
print("Wieder flach:", flattened)
```

Lineare Algebra mit NumPy

```
In [ ]: # Matrixmultiplikation
A = np.array([[1, 2], [3, 4]])
B = np.array([[5, 6], [7, 8]])

dot_product = np.dot(A, B)
print("Matrixmultiplikation:\n", dot_product)

# Determinante
det = np.linalg.det(A)
print("Determinante von A:", det)

# Inverse Matrix (falls möglich)
try:
    inv_A = np.linalg.inv(A)
    print("Inverse von A:\n", inv_A)
except:
    print("Matrix ist nicht invertierbar")
```

Eigenversuch

2a) Erstelle ein 1-Dimensionales Array mit den Zahlen 0-24 und forme es in 5x5 um

```
In [ ]:
```

2b) Berechne die Summe jeder Zeile

In []:

2c) Finde alle Zahlen > 15 (Ausgabe sollte ein 2-Dimensionales Boolean Array sein)

In []:

2d) Berechne den Mittelwert der diagonalen Elemente deiner 5x5 Matrix

In []:

2e) Erstelle eine 3x3 Einheitsmatrix

In []:

2f) Erstelle ein 2D-Array (5x5) mit Zufallszahlen (Floats) zwischen -1 und 1

In []:

2g) Erstelle ein Array mit 1000 zufälligen Integer Werten zwischen 100 und 700

In []:

2h) Berechne für das Integer Array den Median, den Mittelwert, die Standardabweichung, das Minimum und das Maximum

In []:

Aufgabe 3 (Erste Schritte mit Pandas DataFrame)

Pandas ist ein beliebtes Werkzeug, um in Python schnell und effizient Datenanalysen durchzuführen. Die folgenden Zellen enthalten eine interaktive Kurzeinführung. Danach sollen die erlernten Techniken es auf einen neuen Datensatz angewandt werden.

Eine etwas ausführlichere Einführung kann unter https://pandas.pydata.org/docs/user_guide/10min.html gefunden werden.

Weitere nützliche Links sind:

- Pandas Cheat Sheet (https://pandas.pydata.org/Pandas_Cheat_Sheet.pdf)
- Pandas Cookbook (https://pandas.pydata.org/docs/user_guide/cookbook.html)
- Data Wrangling with Pandas (<https://wesmckinney.com/book/>)

In []:

```
# Installation der pandas Bibliothek (falls noch nicht installiert)  
%pip install pandas
```

In []:

```
# Zuerst importieren wir pandas - Konvention ist 'pd' als Alias  
import pandas as pd  
  
# Erstellen eines einfachen DataFrames aus einem Dictionary
```

```
data = {  
    'Name': ['Anna', 'Ben', 'Clara', 'David', 'Eva'],  
    'Alter': [22, 23, 21, 24, 22],  
    'Studienfach': ['Computer Science', 'Business', 'Physics', 'Data Science', '  
    'Note': [1.7, 2.3, 1.0, 2.0, 3.0]  
}  
  
# DataFrame erstellen  
df = pd.DataFrame(data)  
df
```

Daten explorieren

```
In [ ]: # Erste 3 Zeilen anzeigen  
df.head(3)
```

```
In [ ]: # Grundlegende Informationen über den DataFrame  
df.info()
```

```
In [ ]: # Deskriptive Statistiken  
df.describe()
```

Daten auswählen und filtern

```
In [ ]: # Eine einzelne Spalte auswählen (gibt Series zurück)  
df['Name']
```

```
In [ ]: # Mehrere Spalten auswählen  
df[['Name', 'Note']]
```

```
In [ ]: # Zeilen nach Index auswählen  
df.iloc[1:3] # Zeilen 1 und 2 (exklusive 3)
```

```
In [ ]: # Filtern nach Bedingung - nur Informatik-Studierende  
cs_students = df[df['Studienfach'] == 'Computer Science']  
cs_students
```

```
In [ ]: # Durchschnittsnote nach Studienfach  
avg_grade = df.groupby('Studienfach')['Note'].mean()  
avg_grade
```

```
In [ ]: # Mehrere Aggregationen  
df.groupby('Studienfach').agg({  
    'Note': ['mean', 'min', 'max', 'count'],  
    'Alter': 'mean'  
})
```

Mit fehlenden Werten umgehen

```
In [ ]: # DataFrame mit fehlenden Werten erstellen  
data_with_nan = {  
    'Name': ['Anna', 'Ben', 'Clara', 'David'],  
    'Alter': [22, None, 21, 24],  
    'Note': [1.7, 2.3, None, 2.0]
```

```
}  
  
df_nan = pd.DataFrame(data_with_nan)  
df_nan
```

```
In [ ]: # Fehlende Werte identifizieren  
df_nan.isnull()
```

```
In [ ]: # Fehlende Werte ersetzen (hier: Durchschnittsalter)  
average_age = df_nan['Alter'].mean()  
df_nan['Alter'].fillna(average_age, inplace=True)  
df_nan
```

Eigenversuch

3a) Lesen Sie das Housing Prices Dataset (`Housing_Prices.csv`) mithilfe von Pandas ein. Hierzu könnte der folgende Link hilfreich sein:

https://pandas.pydata.org/docs/user_guide/io.html.

```
In [ ]:
```

3b) Zeigen Sie die ersten 3 Datensätze an.

```
In [ ]:
```

3c) Wie viele Zeilen und Spalten hat der Datensatz?

```
In [ ]:
```

3d) Welche Datentypen haben die Spalten?

```
In [ ]:
```

3e) Wie viele Häuser haben eine Klimaanlage?

```
In [ ]:
```

3f) Berechne die grundlegenden statistischen Kennzahlen (Mittelwert, Standardabweichung, Min, Max) für die numerischen Spalten `price`, `area`, `bedrooms`, `bathrooms`

```
In [ ]:
```

3g) Zähle wie viele Häuser:

- Eine Hauptstraßenlage haben (`mainroad` = 'yes') vs. keine
- Ein Gästezimmer haben (`guestroom` = 'yes') vs. keine
- Einen Keller haben (`basement` = 'yes') vs. keine
- Mit Warmwasserheizung (`hotwaterheating` = 'yes') vs. keine

```
In [ ]:
```

3h) Finde alle Häuser für die **alle** folgenden Kriterien gelten:

- Mehr als 3 Schlafzimmern
- Einem Preis unter 12.000.000
- Mindestens 2 Stockwerke
- Mit Klimaanlage

In []:

3i) Erstelle eine neue Spalte `price_per_sqft` die den Preis pro Quadratfuß (`area`) berechnet.

Finde die 5 teuersten Häuser pro Quadratfuß.

Welches Ausstattungsniveau haben die teuersten Häuser?

In []:

Aufgabe 4 (Matplotlib).

Matplotlib ist die wichtigste Plotting-Bibliothek in Python. Sie ermöglicht die Erstellung von vielfältigen Visualisierungen. Im folgenden sind einige Beispiele für die Nutzung. Danach sollen im Eigenversuch Visualisierungen des Housing Prices Datensatzes erstellt werden.

Für weitere Information gibt es die Offizielle Dokumentation, mit Quick Start Guides, Tutorials und Beispielen: <https://matplotlib.org/stable/index.html>

```
In [ ]: import matplotlib.pyplot as plt
import numpy as np
```

Liniendiagramm (Line Plot)

```
In [ ]: x = np.linspace(0, 10, 100) # 100 Werte von 0 bis 10
y1 = np.sin(x)
y2 = np.cos(x)
categories = ['A', 'B', 'C', 'D']
values = [23, 45, 56, 78]

plt.figure(figsize=(10, 5))
plt.plot(x, y1, label='Sinus', color='blue', linewidth=2)
plt.plot(x, y2, label='Cosinus', color='red', linestyle='--')
plt.title('Sinus und Cosinus Funktionen')
plt.xlabel('X-Achse')
plt.ylabel('Y-Achse')
plt.legend()
plt.grid(True)
plt.show()
```

Balkendiagramm (Bar Chart)

```
In [ ]: plt.figure(figsize=(8, 6))
plt.bar(categories, values, color=['red', 'green', 'blue', 'orange'])
plt.title('Balkendiagramm der Werte')
plt.xlabel('Kategorien')
plt.ylabel('Werte')
plt.show()
```

Streudiagramm (Scatter Plot)

```
In [ ]: np.random.seed(42) # Seeden des Random Number Generators für Reproduzierbarkeit
x_scatter = np.random.randn(100) # Normalverteilte Daten
y_scatter = np.random.randn(100)
colors = np.random.rand(100)
sizes = 100 * np.random.rand(100)

plt.figure(figsize=(8, 6))
plt.scatter(x_scatter, y_scatter, c=colors, s=sizes, alpha=0.7, cmap='viridis')
plt.title('Streudiagramm mit Farben und Größen')
plt.colorbar(label='Farbintensität')
plt.show()
```

Histogramm

```
In [ ]: data = np.random.randn(1000) # Normalverteilte Daten

plt.figure(figsize=(8, 6))
plt.hist(data, bins=30, alpha=0.7, color='green', edgecolor='black')
plt.title('Histogramm der Datenverteilung')
plt.xlabel('Werte')
plt.ylabel('Häufigkeit')
plt.show()
```

Tortendiagramm (Pie Chart)

```
In [ ]: plt.figure(figsize=(8, 8))
plt.pie(values, labels=categories, autopct='%1.1f%%', startangle=90)
plt.title('Verteilung der Werte')
plt.show()
```

Subplots (Mehrere Plots in einer Figur)

```
In [ ]: fig, axes = plt.subplots(2, 2, figsize=(12, 10))

# Plot 1
axes[0, 0].plot(x, y1)
axes[0, 0].set_title('Sinus Funktion')

# Plot 2
axes[0, 1].bar(categories, values)
axes[0, 1].set_title('Balkendiagramm')

# Plot 3
axes[1, 0].scatter(x_scatter, y_scatter)
axes[1, 0].set_title('Streudiagramm')
```



```
# Plot 4
axes[1, 1].hist(data, bins=30, edgecolor='black')
axes[1, 1].set_title('Histogramm')

plt.tight_layout()
plt.show()
```

Boxplots

```
In [ ]: np.random.seed(42)
data1 = np.random.normal(100, 15, 200) # Normalverteilung 1
data2 = np.random.normal(120, 20, 200) # Normalverteilung 2
data3 = np.random.normal(90, 10, 200)  # Normalverteilung 3

plt.figure(figsize=(10, 6))
plt.boxplot([data1, data2, data3], labels=['Gruppe A', 'Gruppe B', 'Gruppe C'])
plt.title('Vergleich von drei Datengruppen')
plt.ylabel('Werte')
plt.grid(True, alpha=0.3)
plt.show()
```

Eigenversuch mit dem Housing Prices Datensatz

4a) Erstelle ein Balkendiagramm das zeigt, wie viele Häuser jede Anzahl von Schlafzimmern (`bedrooms`) haben. Beschrifte die Achsen und füge einen Titel hinzu.

In []:

4b) Erstelle ein Histogramm der Preise (`price`). Experimentiere mit verschiedenen Anzahlen von Bins (20, 30, 50). Füge eine Beschriftung hinzu und analysiere die Preisverteilung

In []:

4c) Erstelle ein Diagramm das den durchschnittlichen Preis vergleicht für:

- Häuser mit und ohne Hauptstraßenlage (mainroad)
- Häuser mit und ohne Vorzugslage (prefarea)

Verwende entweder Balkendiagramme oder Boxplots.

In []:

4d) Erstelle ein Streudiagramm von Fläche (`area`) vs. Preis (`price`) Kodiere die Punkte nach der Anzahl der Stockwerke (`stories`) mit verschiedenen Farben füge eine Legende und Farbbar hinzu.

In []:

4e) Erstelle eine 2x2 Subplot Grid mit:

1. Preisverteilung nach Ausstattungstatus (`furnishingstatus`) als Boxplot
2. Durchschnittspreis nach Anzahl Badezimmer als Balkendiagramm
3. Korrelation zwischen Fläche und Preis als Scatter Plot
4. Anteil der Häuser mit Klimaanlage (`airconditioning`) nach Anzahl Stockwerken (`stories`) als Balkendiagramm

In []: